Deterministic_Constraint_Satisfaction_Refinement.companion_notebook.exe (1)

March 27, 2026

1 Deterministic Constraint Satisfaction Refinement — Companion Notebook

This notebook collects the executable components that support the current paper. It is organized around the same computational spine:

1. instrumented SHA-256 as a deterministic state machine
2. algebraic backward walk for a known schedule
3. harmonic and inverse constant experiments
4. metric biomarkers and attractor checks
5. tri-channel decomposition of `T1`, `T2`, and carry exhaust
6. BBP addressing utilities
7. biological and structural bridge functions
8. Samson backpressure and multi-block diagnostics

1.1 1. Constants, rotation primitives, and core operators

The notebook uses the standard SHA-256 initial state and round constants. The harmonic constants `H = /9`, , and `e` are included as analysis parameters for later sections.

[1]:
```
from __future__ import annotations

import math
import struct
import hashlib
from dataclasses import dataclass
from typing import Dict, Iterable, List, Optional, Tuple
```

```
import numpy as np

try:
    import pandas as pd
except Exception:
    pd = None

try:
    import matplotlib.pyplot as plt
except Exception:
    plt = None
```

[2]:
```
M32 = 0xFFFFFFFF
H = math.pi / 9
PHI = (1 + 5 ** 0.5) / 2
E_CONST = math.e

STD_H = [
    0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A,
    0x510E527F, 0x9B05688C, 0x1F83D9AB, 0x5BE0CD19,
]

STD_K = [
    0x428A2F98, 0x71374491, 0xB5C0FBCF, 0xE9B5DBA5, 0x3956C25B, 0x59F111F1,␣
 ↪0x923F82A4, 0xAB1C5ED5,
    0xD807AA98, 0x12835B01, 0x243185BE, 0x550C7DC3, 0x72BE5D74, 0x80DEB1FE,␣
 ↪0x9BDC06A7, 0xC19BF174,
    0xE49B69C1, 0xEFBE4786, 0x0FC19DC6, 0x240CA1CC, 0x2DE92C6F, 0x4A7484AA,␣
 ↪0x5CB0A9DC, 0x76F988DA,
    0x983E5152, 0xA831C66D, 0xB00327C8, 0xBF597FC7, 0xC6E00BF3, 0xD5A79147,␣
 ↪0x06CA6351, 0x14292967,
    0x27B70A85, 0x2E1B2138, 0x4D2C6DFC, 0x53380D13, 0x650A7354, 0x766A0ABB,␣
 ↪0x81C2C92E, 0x92722C85,
    0xA2BFE8A1, 0xA81A664B, 0xC24B8B70, 0xC76C51A3, 0xD192E819, 0xD6990624,␣
 ↪0xF40E3585, 0x106AA070,
    0x19A4C116, 0x1E376C08, 0x2748774C, 0x34B0BCB5, 0x391C0CB3, 0x4ED8AA4A,␣
 ↪0x5B9CCA4F, 0x682E6FF3,
    0x748F82EE, 0x78A5636F, 0x84C87814, 0x8CC70208, 0x90BEFFFA, 0xA4506CEB,␣
 ↪0xBEF9A3F7, 0xC67178F2,
]

H_INV = [(-x) & M32 for x in STD_H]
K_INV = [(-x) & M32 for x in STD_K]

def add32(*xs: int) -> int:
    return sum(xs) & M32
```

```
def rotr(x: int, n: int) -> int:
    return ((x >> n) | (x << (32 - n))) & M32

def big_sigma0(x: int) -> int:
    return rotr(x, 2) ^ rotr(x, 13) ^ rotr(x, 22)

def big_sigma1(x: int) -> int:
    return rotr(x, 6) ^ rotr(x, 11) ^ rotr(x, 25)

def small_sigma0(x: int) -> int:
    return rotr(x, 7) ^ rotr(x, 18) ^ (x >> 3)

def small_sigma1(x: int) -> int:
    return rotr(x, 17) ^ rotr(x, 19) ^ (x >> 10)

def Ch(e: int, f: int, g: int) -> int:
    return ((e & f) ^ ((~e) & g)) & M32

def Maj(a: int, b: int, c: int) -> int:
    return ((a & b) ^ (a & c) ^ (b & c)) & M32
```

1.2 2. Instrumented SHA-256 forward trace

The forward pass is exposed as a full state machine. Each round records the entering registers, the schedule word `W[t]`, the constant `K[t]`, and the two working channels `T1` and `T2`.

[3]:
```
@dataclass
class RoundTrace:
    block: int
    t: int
    a_in: int
    b_in: int
    c_in: int
    d_in: int
    e_in: int
    f_in: int
    g_in: int
    h_in: int
    Wt: int
    Kt: int
    T1: int
    T2: int
    a_out: int
    b_out: int
    c_out: int
    d_out: int
```

```
    e_out: int
    f_out: int
    g_out: int
    h_out: int

@dataclass
class SHA256Trace:
    digest_hex: str
    blocks: List[List[int]]
    schedule: List[List[int]]
    rounds: List[RoundTrace]
    final_state: Tuple[int, ...]
    initial_state: Tuple[int, ...]
    block_feedforward: List[Tuple[Tuple[int, ...], Tuple[int, ...]]]

def pad_sha256(msg: bytes) -> bytes:
    out = bytearray(msg)
    bit_len = len(msg) * 8
    out.append(0x80)
    while len(out) % 64 != 56:
        out.append(0)
    out += struct.pack(">Q", bit_len)
    return bytes(out)

def words32_from_bytes(b: bytes) -> List[int]:
    return list(struct.unpack(f">{len(b)//4}I", b))

def pack_words(words: Iterable[int]) -> bytes:
    return b"".join(struct.pack(">I", w & M32) for w in words)

def digest_words(digest_hex: str) -> List[int]:
    return [int(digest_hex[i:i+8], 16) for i in range(0, 64, 8)]

def expand_schedule(block64: bytes) -> List[int]:
    W = words32_from_bytes(block64)
    for t in range(16, 64):
        W.append(add32(small_sigma1(W[t-2]), W[t-7], small_sigma0(W[t-15]),␣
 ↪W[t-16]))
    return W

def sha256_trace(msg: bytes, H_init: Optional[List[int]] = None, K:␣
 ↪Optional[List[int]] = None) -> SHA256Trace:
    H_init = list(STD_H if H_init is None else H_init)
    K = list(STD_K if K is None else K)
    padded = pad_sha256(msg)
```

```
    blocks = [words32_from_bytes(padded[i:i+64]) for i in range(0, len(padded),␣
↪64)]
    schedules: List[List[int]] = []
    rounds: List[RoundTrace] = []
    block_feedforward: List[Tuple[Tuple[int, ...], Tuple[int, ...]]] = []

    chain = H_init[:]

    for bi in range(0, len(padded), 64):
        block = padded[bi:bi+64]
        W = expand_schedule(block)
        schedules.append(W)

        a, b, c, d, e, f, g, h = chain
        start_state = (a, b, c, d, e, f, g, h)

        for t in range(64):
            T1 = add32(h, big_sigma1(e), Ch(e, f, g), K[t], W[t])
            T2 = add32(big_sigma0(a), Maj(a, b, c))
            a2 = add32(T1, T2)
            e2 = add32(d, T1)

            rounds.append(
                RoundTrace(
                    block=bi // 64,
                    t=t,
                    a_in=a, b_in=b, c_in=c, d_in=d, e_in=e, f_in=f, g_in=g,␣
↪h_in=h,
                    Wt=W[t], Kt=K[t], T1=T1, T2=T2,
                    a_out=a2, b_out=a, c_out=b, d_out=c, e_out=e2, f_out=e,␣
↪g_out=f, h_out=g,
                )
            )

            a, b, c, d, e, f, g, h = a2, a, b, c, e2, e, f, g

        end_working_state = (a, b, c, d, e, f, g, h)
        chain = [add32(x, y) for x, y in zip(chain, end_working_state)]
        block_feedforward.append((start_state, end_working_state))

    digest_hex = "".join(f"{x:08x}" for x in chain)
    return SHA256Trace(
        digest_hex=digest_hex,
        blocks=blocks,
        schedule=schedules,
        rounds=rounds,
        final_state=tuple(chain),
```

```
        initial_state=tuple(H_init),
        block_feedforward=block_feedforward,
    )

def trace_table(trace: SHA256Trace, block_index: int = 0):
    rows = []
    for r in trace.rounds:
        if r.block != block_index:
            continue
        rows.append(
            {
                "t": r.t,
                "W[t]": f"{r.Wt:08x}",
                "T1": f"{r.T1:08x}",
                "T2": f"{r.T2:08x}",
                "a_out": f"{r.a_out:08x}",
                "e_out": f"{r.e_out:08x}",
            }
        )
    return pd.DataFrame(rows) if pd is not None else rows
```

1.3 3. Algebraic un-rotation for a known schedule

For a single block with a known padded block (or known `W[0..63]`), the backward walk is exact. The only inputs required are:

- the digest
- the incoming chaining state (`H_in`)
- the schedule words `W[t]`
- the round constants `K[t]`

No saved forward trace is used.

[4]:
```
@dataclass
class ReverseResult:
    digest_hex: str
    final_working_state: Tuple[int, ...]
    recovered_start_state: Tuple[int, ...]
    states_backward: List[Tuple[int, ...]]
    verified: bool

def subtract_feedforward(digest_hex: str, H_in: Iterable[int] = STD_H) ->␣
 ↪Tuple[int, ...]:
    return tuple((d - h) & M32 for d, h in zip(digest_words(digest_hex), H_in))

def reverse_round_no_trace(after_state: Tuple[int, ...], Wt: int, Kt: int) ->␣
 ↪Tuple[int, ...]:
    a1, b1, c1, d1, e1, f1, g1, h1 = after_state
```

```
    a0 = b1
    b0 = c1
    c0 = d1
    e0 = f1
    f0 = g1
    g0 = h1

    T2 = add32(big_sigma0(a0), Maj(a0, b0, c0))
    T1 = (a1 - T2) & M32
    d0 = (e1 - T1) & M32
    h0 = (T1 - big_sigma1(e0) - Ch(e0, f0, g0) - Kt - Wt) & M32

    return (a0, b0, c0, d0, e0, f0, g0, h0)

def reverse_block_no_trace(
    digest_hex: str,
    *,
    schedule_words: Optional[List[int]] = None,
    padded_block64: Optional[bytes] = None,
    H_in: Iterable[int] = STD_H,
    K: Iterable[int] = STD_K,
) -> ReverseResult:
    H_in = list(H_in)
    K = list(K)

    if schedule_words is None:
        if padded_block64 is None:
            raise ValueError("provide either schedule_words or padded_block64")
        schedule_words = expand_schedule(padded_block64)

    cur = subtract_feedforward(digest_hex, H_in)
    states_backward = [cur]

    for t in range(63, -1, -1):
        cur = reverse_round_no_trace(cur, schedule_words[t], K[t])
        states_backward.append(cur)

    return ReverseResult(
        digest_hex=digest_hex,
        final_working_state=states_backward[0],
        recovered_start_state=cur,
        states_backward=states_backward,
        verified=tuple(H_in) == cur,
    )
```

```
def tail_state_from_digest(digest_hex: str, H_in: Iterable[int] = STD_H) ->␣
 ↪Dict[str, str]:
    fs = subtract_feedforward(digest_hex, H_in)
    labels = ["a63", "b63", "c63", "d63", "e63", "f63", "g63", "h63"]
    return {lab: f"{val:08x}" for lab, val in zip(labels, fs)}
```

1.4 4. Tri-channel ABI decomposition

The forward update `a = T1 + T2` is decomposed into three simultaneously observable channels:

- XOR channel: logical mixing
- Carry channel: overflow scaffold
- Sum channel: modular projection

[5]:
```
def decompose_sum(x: int, y: int) -> Dict[str, int]:
    xor = x ^ y
    carry = x & y
    and2 = (carry << 1) & M32
    summed = (x + y) & M32
    return {
        "xor": xor,
        "carry": carry,
        "and2": and2,
        "sum": summed,
        "carry_pop": carry.bit_count(),
    }

def tri_channel_trace(trace: SHA256Trace, block_index: int = 0):
    rows = []
    for r in trace.rounds:
        if r.block != block_index:
            continue
        dec = decompose_sum(r.T1, r.T2)
        rows.append(
            {
                "t": r.t,
                "carry_pop": dec["carry_pop"],
                "xor": f"{dec['xor']:08x}",
                "carry": f"{dec['carry']:08x}",
                "and2": f"{dec['and2']:08x}",
                "sum": f"{dec['sum']:08x}",
            }
        )
    return pd.DataFrame(rows) if pd is not None else rows

def carry_exhaust_bits(trace: SHA256Trace, block_index: int = 0) -> int:
```

```
    return sum(decompose_sum(r.T1, r.T2)["carry_pop"] for r in trace.rounds if␣
↪r.block == block_index)
```

1.5 5. Harmonic and inverse constant families

These utilities generate structured messages from the SHA constants and related harmonic transforms. They are intended for comparative runs under the standard and inverse pipelines.

[6]:
```
def rotate_words(words: List[int], k: int) -> List[int]:
    k %= len(words)
    return words[k:] + words[:k]

def xor_words(a: List[int], b: List[int]) -> List[int]:
    return [(x ^ y) & M32 for x, y in zip(a, b)]

def sigma_feedback_words(words: List[int]) -> List[int]:
    out = []
    for w in words:
        out.append((small_sigma0(w) ^ small_sigma1(w)) & M32)
    return out

def harmonic_message_family() -> Dict[str, bytes]:
    families: Dict[str, bytes] = {}

    families["K_constants_full"] = pack_words(STD_K[:64])
    families["K_constants_half"] = pack_words(STD_K[:32])
    families["H_constants"] = pack_words(STD_H)
    families["K_INV_as_msg"] = pack_words(K_INV)

    offset = rotate_words(STD_K, 9)
    interleaved = []
    for a, b in zip(STD_K, offset):
        interleaved.extend([a, b])
    families["K_phase_shifted"] = pack_words(interleaved[:128])

    families["K_xor_reverse"] = pack_words(xor_words(STD_K,␣
↪list(reversed(STD_K))))
    families["sigma_feedback"] = pack_words(sigma_feedback_words(STD_K[:32]))

    motif = b"NEXUS|FOLD|HOOK|SCAR|RETURN|glass-key|path-not-mirror|phase|"
    families["structured_harmonic"] = motif * 8

    return families

def signed32(x: int) -> int:
    return x if x < (1 << 31) else x - (1 << 32)
```

```
def trace_energy(trace: SHA256Trace) -> int:
    return sum(abs(signed32((r.T1 - r.T2) & M32)) for r in trace.rounds)

def run_family_comparison(
    H_std: Iterable[int] = STD_H,
    K_std: Iterable[int] = STD_K,
    H_alt: Iterable[int] = H_INV,
    K_alt: Iterable[int] = K_INV,
):
    rows = []
    for name, msg in harmonic_message_family().items():
        t_std = sha256_trace(msg, list(H_std), list(K_std))
        t_alt = sha256_trace(msg, list(H_alt), list(K_alt))
        e_std = trace_energy(t_std)
        e_alt = trace_energy(t_alt)
        rows.append(
            {
                "name": name,
                "bytes": len(msg),
                "blocks": len(t_std.blocks),
                "std_digest": t_std.digest_hex,
                "inv_digest": t_alt.digest_hex,
                "hamming": (int(t_std.digest_hex, 16) ^ int(t_alt.digest_hex,␣
 ↪16)).bit_count(),
                "std_energy": e_std,
                "inv_energy": e_alt,
                "energy_ratio": (e_alt / e_std) if e_std else float("nan"),
            }
        )
    return pd.DataFrame(rows) if pd is not None else rows
```

1.6 6. Biomarkers, attractor checks, and the structural bridge

This section collects three paper-level measurements:

- attractor geometry around H = /9
- digit-frequency and Hamming biomarkers on the constant field
- a lightweight biological / structural bridge through sequence signals and piecewise fold geometry

[7]:
```
def hamming_weight_hex(hx: str) -> int:
    return sum(int(c, 16).bit_count() for c in hx)

def hex_digit_hist_words(words: Iterable[int]) -> Dict[str, int]:
    s = "".join(f"{w:08x}" for w in words)
    return {d: s.count(d) for d in "0123456789abcdef"}

def d_anomaly_count(words: Iterable[int] = STD_K) -> int:
```

```
    return hex_digit_hist_words(words)["d"]

def mark1_geometry() -> Dict[str, float]:
    theta = math.pi / 9
    chord = 2 * math.sin(theta / 2)
    arc = theta
    relative_loss = (arc - chord) / arc
    return {
        "H_radians": theta,
        "H_degrees": math.degrees(theta),
        "arc_length": arc,
        "chord_length": chord,
        "relative_loss": relative_loss,
        "steps_to_circle": round(2 * math.pi / theta),
    }

def first_n_primes(n: int) -> List[int]:
    out: List[int] = []
    x = 2
    while len(out) < n:
        ok = True
        for p in out:
            if p * p > x:
                break
            if x % p == 0:
                ok = False
                break
        if ok:
            out.append(x)
        x += 1
    return out

def prime_root_fraction_stats() -> Dict[str, float]:
    primes = first_n_primes(64)
    fracs = [math.modf(p ** (1 / 3))[0] for p in primes]
    return {
        "mean_fraction": float(np.mean(fracs)),
        "median_fraction": float(np.median(fracs)),
        "std_fraction": float(np.std(fracs)),
        "mean_abs_to_H": float(np.mean([abs(f - H) for f in fracs])),
    }

AA_SCALE = {
    "A": 1.8, "C": 2.5, "D": -3.5, "E": -3.5, "F": 2.8, "G": -0.4, "H": -3.2,
    "I": 4.5, "K": -3.9, "L": 3.8, "M": 1.9, "N": -3.5, "P": -1.6, "Q": -3.5,
    "R": -4.5, "S": -0.8, "T": -0.7, "V": 4.2, "W": -0.9, "Y": -1.3,
}
```

```python
def seq_to_signal(seq: str) -> np.ndarray:
    return np.array([AA_SCALE.get(a, 0.0) for a in seq], dtype=float)

def acf_lag(x: Iterable[float], lag: int = 3) -> float:
    x = np.asarray(list(x), dtype=float)
    if len(x) <= lag:
        return float("nan")
    x = x - x.mean()
    denom = float(np.dot(x, x)) + 1e-12
    return float(np.dot(x[:-lag], x[lag:]) / denom)

def compute_z_sarrus(seq: str) -> float:
    x = seq_to_signal(seq)
    if len(x) < 9:
        return float("nan")
    tri = (x[:-2] + x[1:-1] + x[2:]) / 3.0
    return float(np.mean(np.abs(np.diff(tri))))

def piecewise_fold(labels: str) -> Tuple[np.ndarray, Dict[str, float]]:
    coords = [[0.0, 0.0, 0.0]]
    angle = 0.0
    for lab in labels:
        x, y, z = coords[-1]
        if lab == "H":
            angle += 2 * np.pi / 3.6
            step = np.array([1.5 * np.cos(angle), 1.5 * np.sin(angle), 1.5])
        elif lab == "B":
            step = np.array([3.2, 0.0, (-1) ** len(coords) * 0.4])
        else:
            angle += np.pi / 7
            step = np.array([2.2 * np.cos(angle), 2.2 * np.sin(angle), 0.3 * np.
 ↪sin(angle)])
        coords.append((np.array([x, y, z]) + step).tolist())
    arr = np.array(coords)
    metrics = {
        "points": int(len(arr)),
        "radius_of_gyration": float(np.sqrt(((arr - arr.mean(0)) ** 2).sum(1).
 ↪mean())),
        "z_span": float(np.ptp(arr[:, 2])),
    }
    return arr, metrics
```

1.7 7. BBP addressing and basin view

The BBP routines expose a direct hexadecimal address window into . A basin map is included for state grouping experiments.

[8]:
```
BBP_BASIN_MAP = {
    0: 3, 1: 1, 2: 3, 3: 3, 4: 1, 5: 2, 6: 1, 7: 1,
    8: 1, 9: 2, 10: 2, 11: 3, 12: 3, 13: 1, 14: 1, 15: 3,
}

def bbp_hex_digit_pi(n: int) -> int:
    def series(j: int, n: int) -> float:
        s = 0.0
        for k in range(n + 1):
            s = (s + pow(16, n - k, 8 * k + j) / (8 * k + j)) % 1.0
        t = 0.0
        k = n + 1
        while True:
            new = (16.0 ** (n - k)) / (8 * k + j)
            if abs(new) < 1e-17:
                break
            t += new
            k += 1
        return s + t

    x = (4 * series(1, n) - 2 * series(4, n) - series(5, n) - series(6, n)) % 1.
 ↪0
    return int(16 * x)

def bbp_hex_window(start: int, length: int = 32) -> str:
    return "".join("0123456789ABCDEF"[bbp_hex_digit_pi(start + i)] for i in␣
 ↪range(length))

def bbp_basins(start: int, length: int = 32) -> List[int]:
    return [BBP_BASIN_MAP[int(c, 16)] for c in bbp_hex_window(start, length)]
```

1.8 8. Samson backpressure and multi-block boundary diagnostics

The functions below give a simple computational interface for the paper's backpressure language. They do not solve the general multi-block inversion problem, but they provide the local diagnostics needed to instrument those boundaries.

[9]:
```
def samson_coefficient(delta: float, tau: float, h: float = H) -> float:
    tau = max(float(tau), 1e-9)
    return float((delta / tau) * h)

def delta_attraction(delta: float, tau: float, h: float = H) -> Dict[str, float␣
 ↪| str]:
    s = samson_coefficient(delta, tau, h)
    if s > h:
        regime = "damping"
    elif s < h:
```

```
        regime = "amplification"
    else:
        regime = "phase_lock"
    return {"S": s, "regime": regime}

def schedule_consistency(W: List[int]) -> Dict[str, int | bool]:
    errors = []
    for t in range(16, 64):
        want = add32(small_sigma1(W[t-2]), W[t-7], small_sigma0(W[t-15]),␣
 ↪W[t-16])
        errors.append((W[t] - want) & M32)
    return {"consistent": all(e == 0 for e in errors), "nonzero_errors": sum(e !
 ↪= 0 for e in errors)}

def last_block_boundary_diagnostic(msg: bytes):
    trace = sha256_trace(msg)
    if len(trace.blocks) < 2:
        return {"blocks": len(trace.blocks), "note": "single-block message"}

    prev_in, prev_end = trace.block_feedforward[-2]
    last_in, _ = trace.block_feedforward[-1]
    derived = tuple(add32(x, y) for x, y in zip(prev_in, prev_end))

    return {
        "blocks": len(trace.blocks),
        "last_block_chaining_matches_prev": derived == last_in,
        "last_block_input_state": [f"{x:08x}" for x in last_in],
        "derived_from_prev": [f"{x:08x}" for x in derived],
    }
```

1.9 9. Reproducible demonstrations

The cells below execute the major components of the notebook on small, self-contained examples.

[10]:
```
demo_msg = b"NEXUS|FOLD|HOOK|SCAR|RETURN|"
demo_trace = sha256_trace(demo_msg)

print("message bytes =", len(demo_msg))
print("digest        =", demo_trace.digest_hex)
print("hashlib match =", demo_trace.digest_hex == hashlib.sha256(demo_msg).
 ↪hexdigest())

trace_table(demo_trace).head(8) if pd is not None else trace_table(demo_trace)[:
 ↪8]
```

```
message bytes = 28
digest        = 4fd9e99c31139dad682094dc67a3a9d6d27a3b64fe30fe79d579c44e149fd75e
hashlib match = True
```

[10]:

	t	W[t]	T1	T2	a_out	e_out
0	0	4e455855	41bd45bd	08909ae5	4a4de0a2	e70d3af7
1	1	537c464f	454165c9	0a518a15	4f92efde	81b0593b
2	2	4c447c48	7f7365c9	f0b3a544	70270b0d	3adb144e
3	3	4f4f4b7c	2c4dc489	e25661c9	0ea42652	9657aaf0
4	4	53434152	3f397c04	704663ed	af7fdff1	89875ca6
5	5	7c524554	d7ac3845	99545a10	71009255	273f2823
6	6	55524e7c	cb207b36	fbc78fa6	c6e80adc	3b478643
7	7	80000000	cc44b9e0	aedfe1c1	7b249ba1	dae8e032

[11]:
```
single_block_msg = b"NEXUS|HOOK|SCAR|RETURN|PHASE|ECHO|SHIFT|GLASS|KEY|H=PI/9"[:
 ↪55]
single_trace = sha256_trace(single_block_msg)
single_rev = reverse_block_no_trace(
    single_trace.digest_hex,
    padded_block64=pad_sha256(single_block_msg),
)

print("single-block bytes =", len(single_block_msg))
print("single-block digest =", single_trace.digest_hex)
print("tail state          =", tail_state_from_digest(single_trace.digest_hex))
print("reverse verified    =", single_rev.verified)
print("recovered start == IV =", single_rev.recovered_start_state ==␣
 ↪tuple(STD_H))
```

```
single-block bytes = 55
single-block digest =
01dfa391badd78914b59a61a1b982a1d9ddd874a8a175279dc83dd325c93b148
tail state          = {'a63': '97d5bd2a', 'b63': 'ff75ca0c', 'c63': '0eeab2a8',
'd63': '764834e3', 'e63': '4ccf34cb', 'f63': 'ef11e9ed', 'g63': 'bd000387',
'h63': '00b2e42f'}
reverse verified    = True
recovered start == IV = True
```

[12]:
```
comparison = run_family_comparison()
comparison
```

[12]:

	name	bytes	blocks	\
0	K_constants_full	256	5	
1	K_constants_half	128	3	
2	H_constants	32	1	
3	K_INV_as_msg	256	5	
4	K_phase_shifted	512	9	
5	K_xor_reverse	256	5	
6	sigma_feedback	128	3	
7	structured_harmonic	480	8	

	std_digest	\

```
0  e4c62ae41929873b99fa3f871694451c9dbd0300448aba…
1  d2c917c55c86db7769a169d0ee5b8bf63428307d11168e…
2  5cce668898a2e2d7dedbb552165a2a77596d092b03df3f…
3  25128c8cc6689732e068c5209f2ec3bf3d8ce122617f5a…
4  eee018930c2cb7aacacc6ff72b792dcc067ff5ac6673a2…
5  8b8e219f0d987c0a578da684124fae28a4940f7f8de30a…
6  8bd5c310f00358d172a78ebef80ae744114152dab33fc0…
7  23db38ca7ddab7ffc2d531f4722c4818ab4ff1d1368a8a…

                                          inv_digest  hamming    std_energy  \
0  5f80e54690df0f0cfb86a07823db4ff973eec9c4031d6f…      146  337551798143
1  3684b8cc69bb55982ddc7e6540d064b0ead01bac154905…      139  208304802809
2  a79262642290e355c68887896a2aa043377899ab1a0b8a…      126   69703670393
3  e5748049c125a3491d00453b562f094be739c2319f601b…      124  341781666420
4  4949402d012aecebb6cae0a713aece2b2c8a0c46fd3dee…      132  636818515842
5  8af3916b93dee79af6ddb617c660d11a28a99ee3cc3cfe…      126  366246717349
6  9b453ec1a80ec2e77dd5ccae901b3b768afdad3661c809…      128  193687931188
7  70a47abf3599936f25bfb598194c0bd1ed3ee2c5bf0fce…      110  542880021707

     inv_energy  energy_ratio
0  352482160034      1.044231
1  211607365535      1.015854
2   66358460531      0.952008
3  336898936169      0.985714
4  635387346711      0.997753
5  344819550255      0.941495
6  199788441072      1.031497
7  570940093248      1.051687
```

[13]:
```
biomarkers = {
    "mark1_geometry": mark1_geometry(),
    "d_anomaly_count": d_anomaly_count(STD_K),
    "constant_digit_histogram": hex_digit_hist_words(STD_K),
    "prime_root_fraction_stats": prime_root_fraction_stats(),
}
biomarkers
```

[13]:
```
{'mark1_geometry': {'H_radians': 0.3490658503988659,
  'H_degrees': 20.0,
  'arc_length': 0.3490658503988659,
  'chord_length': 0.34729635533386066,
  'relative_loss': 0.005069229954701356,
  'steps_to_circle': 18},
 'd_anomaly_count': 19,
 'constant_digit_histogram': {'0': 31,
  '1': 38,
  '2': 33,
```

```
  '3': 25,
  '4': 32,
  '5': 31,
  '6': 31,
  '7': 36,
  '8': 38,
  '9': 31,
  'a': 36,
  'b': 34,
  'c': 43,
  'd': 19,
  'e': 25,
  'f': 29},
 'prime_root_fraction_stats': {'mean_fraction': 0.4777901391168488,
  'median_fraction': 0.46800936127245096,
  'std_fraction': 0.262102258394429,
  'mean_abs_to_H': 0.24551657362573803}}
```

[14]:
```
tri_df = tri_channel_trace(single_trace)
print("carry exhaust bits (block 0) =", carry_exhaust_bits(single_trace, 0))
tri_df.head(10) if pd is not None else tri_df[:10]
```

```
carry exhaust bits (block 0) = 518
```

[14]:

	t	carry_pop	xor	carry	and2	sum
0	0	6	492ddf58	009000a5	0120014a	4a4de0a2
1	1	4	4f10eddc	00410201	00820402	4f92f1de
2	2	9	62a0ff08	805b00c4	00b60188	63570090
3	3	6	cbafa847	000053a0	0000a740	cbb04f87
4	4	14	66246740	995a90a3	32b52146	98d98886
5	5	6	3de937b9	4210c004	84218008	c20ab7c1
6	6	8	c14ce9a5	2ea00210	5d400420	1e8cedc5
7	7	8	bb7b560b	04848930	09091260	c484686b
8	8	11	c0bd170f	2a4268e0	5484d1c0	1541e8cf
9	9	7	a9c463e7	10109418	20212830	c9e58c17

[15]:
```
print("first 32 BBP hex digits of  =", bbp_hex_window(0, 32))
print("basin path for first 32 digits =", bbp_basins(0, 32))
```

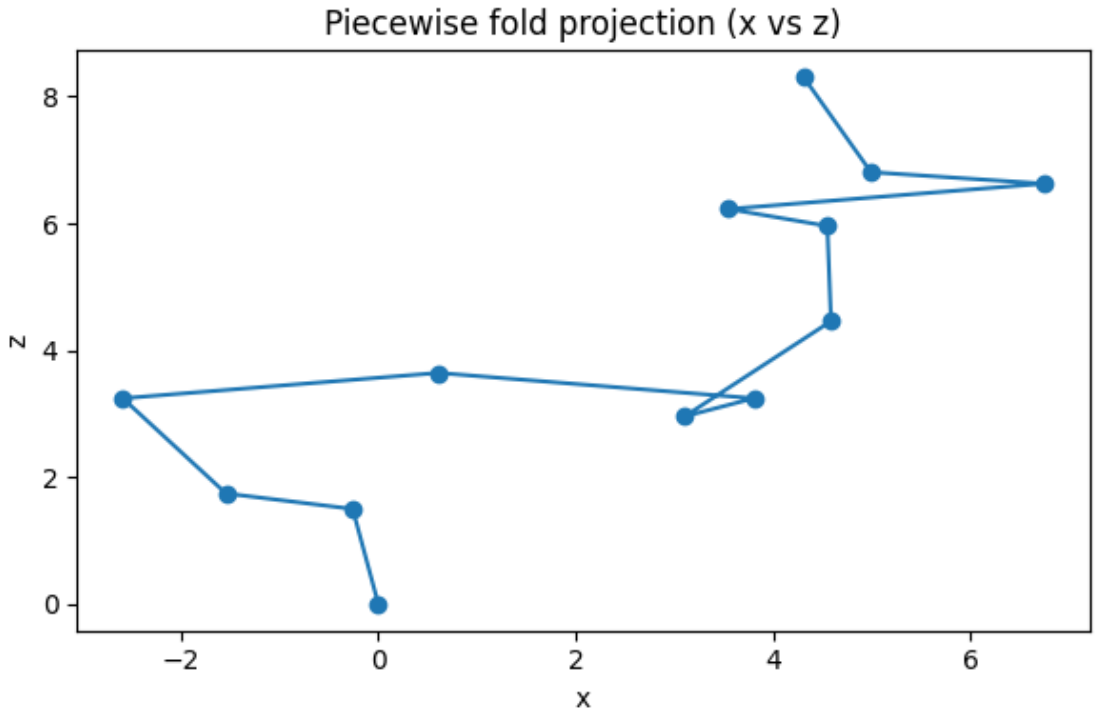
```
first 32 BBP hex digits of  = 243F6A8885A308D313198A2E03707344
basin path for first 32 digits = [3, 1, 3, 3, 1, 2, 1, 1, 1, 2, 2, 3, 3, 1, 1,
3, 1, 3, 1, 2, 1, 2, 3, 1, 3, 3, 1, 3, 1, 3, 1, 1]
```

[16]:
```
sequence = "ACDEFGHIKLMNPQRSTVWY"
signal = seq_to_signal(sequence)
labels = "HCHBBCHHCBCH"
coords, fold_metrics = piecewise_fold(labels)
```

```
print("sequence acf lag-3 =", acf_lag(signal, lag=3))
print("z_sarrus =", compute_z_sarrus(sequence))
print("piecewise fold metrics =", fold_metrics)

if plt is not None:
    plt.figure(figsize=(6, 4))
    plt.plot(coords[:, 0], coords[:, 2], marker="o")
    plt.title("Piecewise fold projection (x vs z)")
    plt.xlabel("x")
    plt.ylabel("z")
    plt.tight_layout()
    plt.show()
```

```
sequence acf lag-3 = 0.11848712231149425
z_sarrus = 0.9607843137254901
piecewise fold metrics = {'points': 13, 'radius_of_gyration':
3.8955183897231294, 'z_span': 8.3073060636057}
```



[17]:
```
multi_msg = harmonic_message_family()["structured_harmonic"]
print(last_block_boundary_diagnostic(multi_msg))
print("schedule consistency on last block =",␣
  ↪schedule_consistency(sha256_trace(multi_msg).schedule[-1]))
print("delta attraction example =", delta_attraction(delta=1.25, tau=0.8))
```

```
{'blocks': 8, 'last_block_chaining_matches_prev': True,
'last_block_input_state': ['5f4cacc8', 'f7378d0c', '57642980', 'c1315ef1',
'c32755a1', '74cf4247', '5e939798', 'cdea6666'], 'derived_from_prev':
['5f4cacc8', 'f7378d0c', '57642980', 'c1315ef1', 'c32755a1', '74cf4247',
'5e939798', 'cdea6666']}
schedule consistency on last block = {'consistent': True, 'nonzero_errors': 0}
delta attraction example = {'S': 0.5454153912482279, 'regime': 'damping'}
```

1.10 10. Working note

This notebook is designed as a computational appendix. The functions are intended to be imported, rerun, or extended directly from the paper context.